ZeroSecureCTF 2026

Detailed Flags and Solutions

Author: anilog

Prepared for the offline challenge pack generated for CTF7 upload.
This document contains the final flags plus step-by-step solution paths for all 10 selected categories.

Categories covered: Cryptography, Web Security, Reverse Eng, Forensics, Network Sec, OSINT, Blockchain, AI & ML, Miscellaneous, Surprise.

1. Reused Signal

Category: Cryptography | Points: 100 | Author: anilog | Files: transmissions.txt

Flag: ZeroSecureCTF{never_reuse_xor_keys}

Overview. This is a classic repeated-key XOR failure. The file gives one plaintext/ciphertext pair and a second ciphertext encrypted with the same repeating key. Once the keystream is recovered from the known pair, the second message falls immediately.

Solve Steps.

- Open transmissions.txt. The first line gives a full known plaintext: status=greenlight::ok. The next two lines are ciphertexts in hex.
- Convert frame_01_ciphertext_hex from hex to bytes and XOR it against the known plaintext bytes. That directly recovers the keystream bytes used for those positions.
- Because the note says the same stream seed is reused every 6 bytes, inspect the recovered stream for periodicity. The shortest repeating unit is ORBIT5. That is the repeated XOR key.
- Now decode frame_02_ciphertext_hex from hex and XOR it with the repeating key ORBIT5 across the full length.
- The decrypted plaintext is flag=ZeroSecureCTF{never_reuse_xor_keys}. Strip the flag= prefix and submit the flag.

Useful Commands / Example Solve Snippets.

```
python3 - <<'PY'
from itertools import cycle
pt = b'status=greenlight::ok'
ct1 = bytes.fromhex('<frame_01_hex>')
ct2 = bytes.fromhex('<frame_02_hex>')
stream = bytes(a ^ b for a, b in zip(pt, ct1))
key = b'ORBIT5'
print(bytes(a ^ b for a, b in zip(ct2, cycle(key))).decode())
PY
```

Why this works. *Known-plaintext plus key reuse is fatal for stream ciphers and repeated-key XOR. The challenge is introductory but forces the solver to notice the repetition instead of brute-forcing.*

2. Backup Appetite

Category: Web Security | Points: 110 | Author: anilog | Files: admin-portal.zip

Flag: ZeroSecureCTF{dont_ship_debug_backups}

Overview. The challenge is intentionally offline. Instead of running a live target, the participant inspects a leaked source bundle and notices that an old backup route still exposes sensitive data. The flag is stored in an encoded audit file inside the archive.

Solve Steps.

- Unzip admin-portal.zip and review the source tree. The current route in routes/export.js only returns a 403 JSON error, which is there to misdirect casual readers.
- Keep digging into the archive and inspect the backups directory. The file backups/export.old.js contains a previous implementation of the export endpoint.
- That old route checks for a debug token and then reads storage/audit.json. Even though the route is not wired into the current app, the backup file reveals where the sensitive data lives.
- Open storage/audit.json. It contains a key named flag_b64. The value is a Base64-encoded flag.
- Decode the Base64 string. The result is ZeroSecureCTF{dont_ship_debug_backups}. Submit that exact value.

Useful Commands / Example Solve Snippets.

```
unzip admin-portal.zip -d portal
```

```
grep -R "flag_b64\|debugToken\|audit.json" -n portal
```

```
python3 - <<'PY'
import base64, json
obj = json.load(open('portal/storage/audit.json'))
print(base64.b64decode(obj['flag_b64']).decode())
PY
```

Why this works. *Backups, legacy routes, and forgotten debug utilities routinely leak secrets. Shipping source snapshots inside attachments is enough to create a realistic web challenge without requiring live hosting.*

3. License Whisper

Category: Reverse Eng | Points: 120 | Author: anilog | Files: LicenseWhisper.jar

Flag: ZeroSecureCTF{java_bytecode_is_chatty}

Overview. This is a lightweight reversing problem. The JAR asks for a single accepted input. The program contains the validation logic directly inside the bytecode, so the flag can be recovered with any Java decompiler or even by reading the constant arrays from the class file.

Solve Steps.

- Run the JAR once if desired; it prompts with license> and prints accepted or denied. There is no need to brute-force the input.
- Decompile the class with jadx, cfr, javap -c, or another Java reverse-engineering tool. The reveal() method is the key.
- Inside reveal(), the program iterates over an integer array named DATA. For each index i, it computes a mask as 17 + (i % 7) * 3 and XORs the array value with that mask to reconstruct a character.
- Reimplement the same loop in a short script or even by hand for the first few bytes. Concatenating the decoded characters produces the full required license string.
- The reconstructed string is ZeroSecureCTF{java_bytecode_is_chatty}. Feeding it back into the JAR returns accepted, confirming the solution.

Useful Commands / Example Solve Snippets.

```
javap -classpath LicenseWhisper.jar -c LicenseWhisper
```

```
python3 - <<'PY'
DATA = [list values from the decompiler]
out = []
for i, v in enumerate(DATA):
    mask = 17 + (i % 7) * 3
    out.append(chr(v ^ mask))
print(''.join(out))
PY
```

Why this works. *Entry-level Java reversing is usually about extracting constants and replicating short decoding logic. The challenge rewards basic decompilation literacy rather than advanced obfuscation handling.*

4. Trailing Memory

Category: Forensics | Points: 110 | Author: anilog | Files: family_trip.jpg

Flag: ZeroSecureCTF{jpgs_can_hide_zips}

Overview. The JPEG is valid, but extra data was appended after the normal image end marker. This is a standard file-carving and appended-archive problem intended to be approachable with either automated tools or manual inspection.

Solve Steps.

- Start by checking the file type. It opens as an ordinary JPEG, which encourages the solver to think about metadata, stego, or appended data.
- Run binwalk, foremost, or inspect the file tail with a hex viewer. You will find ZIP signatures after the JPEG image has already ended.
- Because ZIP archives can be appended to arbitrary files and still remain recoverable, extract the embedded archive directly from the JPG. Most ZIP tools can read the appended ZIP without needing you to carve bytes manually.
- Inside the recovered archive there is a file named recovered.txt. Open it to reveal the flag.
- The flag inside is ZeroSecureCTF{jpgs_can_hide_zips}. Submit it as-is.

Useful Commands / Example Solve Snippets.

```
binwalk family_trip.jpg
7z x family_trip.jpg
unzip family_trip.jpg
```

Why this works. *Appended archives are a common beginner forensics pattern. The challenge teaches solvers to distrust file extensions and to inspect trailing data whenever a file looks too normal.*

5. Silent TXT

Category: Network Sec | Points: 130 | Author: anilog | Files: silent-txt.pcap

Flag: ZeroSecureCTF{dns_can_smuggle_more_than_ips}

Overview. The packet capture includes ordinary-looking traffic plus a small DNS exfiltration channel. The hidden payload is split across TXT-query subdomains and has to be reordered and decoded.

Solve Steps.

- Open the PCAP in Wireshark or tshark and review the DNS traffic. Most of the packets are noise, but a subset of queries go to sync.zs-telemetry.net. Those are the interesting ones.
- Notice that each suspicious query begins with a numeric prefix such as 01. and 02. The next label in each query contains a chunk of lowercase Base32 text.
- Filter only the TXT queries to that domain. Extract the chunk labels in numeric order, concatenate them, and add Base32 padding if your decoder expects it.
- Decode the reconstructed Base32 string. The output is the flag: ZeroSecureCTF{dns_can_smuggle_more_than_ips}.
- Submit the decoded flag. The point of the challenge is the identification and reconstruction of the covert channel, not any protocol exploitation.

Useful Commands / Example Solve Snippets.

```
tshark -r silent-txt.pcap -Y "dns.qry.type == 16" -T fields -e dns.qry.name
```

```
python3 - <<'PY'
import base64
chunks = [('01','...'), ('02','...')]  # replace with extracted values
payload = ''.join(v for _, v in sorted(chunks))
payload += '=' * ((8 - len(payload) % 8) % 8)
print(base64.b32decode(payload.upper()).decode())
PY
```

Why this works. *DNS is frequently abused for covert transfer because the traffic blends into normal infrastructure chatter. The challenge is intentionally small so the solver focuses on recognizing structure in query names.*

6. Badge Trail

Category: OSINT | Points: 125 | Author: anilog | Files: forum_post.txt, commit_snippet.csv, metro-card.png

Flag: ZeroSecureCTF{lakdikapulnight26}

Overview. This is an offline OSINT-style correlation problem. Instead of using the internet, the participant receives the sort of public crumbs a person might leave behind and has to derive the reused username from a stated pattern.

Solve Steps.

- Read forum_post.txt. The author explicitly says the handle is always constructed from three parts: metro stop without separators, best working time of day, and graduating year.
- Now inspect the other provided artifacts for those three parts. metro-card.png gives the station as Lakdi-ka-pul, which becomes lakdikapul after removing punctuation and spaces.
- The commit history in commit_snippet.csv includes the phrase night shift notes cleanup, strongly indicating the time-of-day component is night.
- The image also states Valid till: Jul 2026, and the commits mention the 2026 batch, so the graduation year piece is 26.
- Join the three pieces exactly as described: lakdikapul + night + 26. Wrap that handle in the event flag format to obtain ZeroSecureCTF{lakdikapulnight26}.

Why this works. *Good OSINT is often pattern recognition and cross-correlation rather than deep tooling. This challenge turns that idea into a self-contained puzzle suitable for offline deployment.*

7. Private Means Private?

Category: Blockchain | Points: 130 | Author: anilog | Files: Vault.sol, storage_dump.txt

Flag: ZeroSecureCTF{private_vars_live_on_chain}

Overview. The point here is to teach a foundational blockchain misconception: Solidity private variables are not secret. They are only inaccessible to other contracts through the language interface, but they still sit in chain storage and can be read off-chain.

Solve Steps.

- Open Vault.sol and inspect the state layout. The contract defines owner, attempts, and then two bytes32 private values: rescueCodePart1 and rescueCodePart2.
- Read storage_dump.txt. It already shows the raw values returned by eth_getStorageAt for slots 0 through 3. Slots 2 and 3 correspond to the two private bytes32 variables.
- Interpret the hex from slots 2 and 3 as ASCII bytes, trimming trailing null bytes. Slot 2 gives the first part of the flag string and slot 3 gives the remainder.
- Concatenate the decoded strings in order. The combined value becomes ZeroSecureCTF{private_vars_live_on_chain}.
- Submit the reconstructed flag. No contract interaction is required; the entire solve comes from understanding storage visibility.

Useful Commands / Example Solve Snippets.

```
python3 - <<'PY'
slot2 = '<hex from slot 2>'
slot3 = '<hex from slot 3>'
print(bytes.fromhex(slot2).rstrip(b'\x00').decode() + bytes.fromhex(slot3).rstrip(b'\x00').decode())
PY
```

Why this works. *On-chain privacy does not exist just because the source code says private. The distinction between visibility and confidentiality is the entire lesson.*

8. Tokenizer Graveyard

Category: AI & ML | Points: 140 | Author: anilog | Files: tokenizer.json, generation.log

Flag: ZeroSecureCTF{tokenizers_remember_debug_artifacts}

Overview. The challenge models a realistic ML engineering mistake: debug-only tokens were left inside a tokenizer export, and a log captured the exact special-token sequence used during an internal generation. That sequence is enough to reconstruct a hidden payload.

Solve Steps.

- Open generation.log. The key field is special_token_trace, which lists several token IDs joined with arrows.
- Next, inspect tokenizer.json. Under added_tokens, some special tokens have content shaped like <dbg_...>. Those are the pieces of the hidden payload.
- Map each token ID from the trace to its corresponding content in added_tokens. Strip the surrounding <dbg_ prefix and trailing > so only the encoded payload segments remain.
- Concatenate the segments in the order given by the trace. The joined string is URL-safe Base64 without padding.
- Add padding if needed and decode the result. The decoded output is ZeroSecureCTF{tokenizers_remember_debug_artifacts}.

Useful Commands / Example Solve Snippets.

```
python3 - <<'PY'
import base64, json
trace = [50031, 50009, 50044]  # replace with the ids from generation.log
obj = json.load(open('tokenizer.json'))
lookup = {t['id']: t['content'] for t in obj['added_tokens']}
joined = ''.join(lookup[i][5:-1] for i in trace)
joined += '=' * ((4 - len(joined) % 4) % 4)
print(base64.urlsafe_b64decode(joined).decode())
PY
```

Why this works. *ML artifacts often preserve more operational detail than teams expect. The challenge frames tokenizer exports and debug traces as a security surface rather than a purely ML concern.*

9. Hold Music

Category: Miscellaneous | Points: 115 | Author: anilog | Files: hold-music.wav

Flag: ZeroSecureCTF{morse_still_gets_through}

Overview. This is an audio-encoding puzzle. The WAV is not encrypted; it encodes a text phrase through tone duration and silence spacing using Morse code timing. Once the phrase is recovered, the flag format is straightforward.

Solve Steps.

- Open the WAV in an audio editor or listen closely. You will hear short and long beeps instead of speech or music. That is the immediate hint that the content is Morse timing.
- Measure the durations of tones and the silences between them. Short beeps represent dots, long beeps represent dashes, short gaps separate symbols, and longer gaps separate letters.
- Transcribe the Morse into characters. The decoded phrase is morse_still_gets_through. The file does not directly include the full flag wrapper.
- The challenge statement or event convention tells you the flag format is ZeroSecureCTF{...}. Wrap the recovered phrase in that format.
- The final answer is ZeroSecureCTF{morse_still_gets_through}.

Why this works. *Not every misc challenge needs obscure gimmicks. A clean signal-interpretation task is enough if the encoding and solve path are both crisp.*

10. Invisible Ink

Category: Surprise | Points: 145 | Author: anilog | Files: status-memo.txt

Flag: ZeroSecureCTF{zero_width_marks_the_drop}

Overview. The visible text is a decoy. The real payload is hidden entirely in zero-width Unicode characters appended to the memo. Solvers who inspect the text programmatically or in a Unicode-aware editor can recover the binary stream.

Solve Steps.

- Open status-memo.txt. The visible memo looks harmless and even hints that there is more here than what the eye first catches. That points toward invisible characters rather than visible steganography.
- Use a script or a Unicode-aware tool to extract characters in the zero-width range. Three code points are used here: zero-width space, zero-width non-joiner, and zero-width joiner.
- Map zero-width space to bit 0 and zero-width non-joiner to bit 1. Treat zero-width joiner as the separator between bytes.
- Split the hidden stream on the separator, convert each 8-bit binary chunk into a byte value, and decode the resulting byte string as ASCII.
- The recovered text is ZeroSecureCTF{zero_width_marks_the_drop}. Submit it directly.

Useful Commands / Example Solve Snippets.

```
python3 - <<'PY'
text = open('status-memo.txt', encoding='utf-8').read()
hidden = ''.join(c for c in text if c in '\u200b\u200c\u200d')
out = []
for chunk in hidden.split('\u200d'):
    bits = ''.join('0' if c == '\u200b' else '1' for c in chunk)
    out.append(int(bits, 2))
print(bytes(out).decode())
PY
```

Why this works. *Invisible Unicode characters are easy to miss in reviews, logs, and copied text. The challenge gives a simple but memorable introduction to that class of hiding technique.*